Pickleball Session Generator - Chat Transcript

Pickleball Session Generator — Chat Transcript

A record of the Claude Code session that built this project.

---

1. Initial request: `PickleballRound` class

User: I want to create a python class that represents a pickleball session round. There are 8 players, represented by A-H and two courts. They should randomly paired into doubles teams, randomly assigned to either court 1 or court 2 and then one of the teams on each court should be randomly selected to serve.

The python class needs no constructor, just a generate() method that creates it. It needs a comparison operator. 2 rounds are equal if the same 4 players are together on the court, regardless of teams or who is serving. There also needs to be a repr function that prints the round pairings using ASCII art so it is easily viewable

Result: Created `pickleball_round.py` with `PickleballRound`: - `generate()` shuffles 8 players (`A`-`H`), pairs them into 4 doubles teams, assigns 2 teams to each of 2 courts, and randomly picks a serving team per court. - `__eq__` / `__hash__` compare rounds by the **set of 4 players per court**, ignoring team pairing and server. - `__repr__` renders each court as an ASCII box, marking the serving team with `*`.

---

2. Test script

User: write me a test script i can use to generate and print a round

Result: Added `test_pickleball_round.py` — generates one round and prints it.

3. `PickleballSession` class

User: Each session is N rounds. Write a new class that PickleballSession that has N rounds (default 6). Each round must be unique by the rules of the PickeballRound equals function. Again is should have a repr function that prints the whole session. if possible, this should be printed compactly horizontally

Result: - Refactored `PickleballRound` to expose `render_lines(title=None)` so a session can compose multiple rounds' ASCII art. - Created `pickleball_session.py` with `PickleballSession.generate(num_rounds=6)`, which rejection-samples rounds until `num_rounds` unique ones (by `PickleballRound.__eq__`) are collected. - `__repr__` lays all rounds' ASCII boxes side-by-side horizontally, padded to equal height/width.

---

4. Private constructors, classmethod `generate()`

User: Can you make the constructors for both private and the generate function should be a class function

Result: - Both `PickleballRound.__init__` and `PickleballSession.__init__` now raise `TypeError`, directing callers to use `generate()`. - `generate()` became a `@classmethod` on both classes, using `object.__new__(cls)` to build the instance. - Updated all call sites (`test_pickleball_round.py`, `test_pickleball.py`, `__main__` blocks) from `PickleballRound()` / `PickleballSession()` to `PickleballRound.generate()` / `PickleballSession.generate()`.

---

5. Uniqueness/variety heuristic score

User: Add a heuristic to PickleballSession. It should score sessions higher if players switch courts, partners, and serving more often to maximize "uniqueness"

Result: - Added `PickleballRound.player_details()` — per-player `(court, partner, is_serving)` for that round. - Added `PickleballSession.score()`: - Builds each player's timeline of `(court, partner, is_serving)` across all rounds. - For each dimension

(court, partner, serving), computes a **variety** score = average of: - **spread**: normalized Shannon entropy (0–1, 1.0 = as evenly distributed as possible given sequence length) - **switch rate**: fraction of adjacent rounds where the value changed - Combines the three dimensions as a weighted average (partner weighted 2×, court and serve 1× by default). - Verified: a session of 6 identical rounds scores `0.000`; sessions with full variety score `1.000`; random sessions land ~0.64–0.86.

User (follow-up): explain the normalized entropy equation

Explanation given:

```
H = −Σ pᵢ · ln(pᵢ)              # entropy, in nats
spread = H / ln(achievable)      # normalize to 0..1
```

- `achievable = min(num_options, len(sequence))` — normalizes against the best a sequence of *this length* could actually do (e.g. 6 rounds can't visit 7 possible partners, so the ceiling is 6, not 7).
- Entropy alone is **order-blind** (`1,1,1,2,2,2` and `1,2,1,2,1,2` score identically), which is why it's averaged with `switch_rate` to also reward round-to-round change.

---

6. Configurable weights

User: Allow weights to be passed in to the score function as named arguments, if it's not provided use the defaults in the class

Result: - Moved `COURT_WEIGHT`, `PARTNER_WEIGHT`, `SERVE_WEIGHT` from module constants to class attributes on `PickleballSession` (so subclasses can override defaults). - `score(court_weight=None, partner_weight=None, serve_weight=None)` — each `None` falls back to the class attribute. - Raises `ValueError` if all effective weights are ≤ 0 (would divide by zero).

---

7. Combinatorics question

User: given the 6 rounds, 8 players, 2 courts, and binary server or not, how many possible grids are there (ignoring the uniqueness of rounds constraint)

Answer: 2,520 distinct single-round configurations: 1. Split 8 players into two labeled groups of 4: `C(8,4) = 70` 2. Pair each court's 4 into 2 teams: 3 ways per court → `3 × 3 = 9` 3. Pick the serving team per court: `2 × 2 = 4` - Total: `70 × 9 × 4 = 2,520` - Noted separately: under `PickleballRound.__eq__` (court-unordered, ignoring teams/server), there are only **35** truly distinct rounds (`70 / 2`) — a hard ceiling worth knowing since `PickleballSession.generate()` uses rejection sampling.

---

8. `statistics.fmean` cleanup

User: do we need to redefine mean() is there no built in or math function that does that?

Result: Replaced the hand-rolled `_mean()` helper with `statistics.fmean` from the stdlib.

---

9. `best_session.py` script

User: Write a new script that generates N sessions and picks the best session by heuristic score. N should be a positional argument, default 1000, there should be optional arguments for each weight as well. the final "best" session is printed with the score rounded to 3 digits

Result: Created `best_session.py`:

```
python3 best_session.py [N] [--court-weight W] [--partner-weight W] [--
serve-weight W]
```

(later the user manually edited flags to short forms `-c` , `-p` , `-s` .) Generates `N` sessions, keeps the highest-scoring one, prints it with `score = {best_score:.3f}` .

---

10. Interactive mode for non-CLI users

User: Add interactivity to the best_session.py if no arguments are given. First question: customize options? → then how many sessions → then customize weights? → then each weight in turn on a scale from 1-3.

Result: When run with **no CLI arguments**, `best_session.py` now prompts interactively: 1. `Customize options? [y/N]` 2. If yes: `How many sessions to generate? [1000]` 3. `Customize weights? [y/N]` 4. If yes: `Court weight, 1–3 [default]`, `Partner weight, 1–3 [default]`, `Serve weight, 1–3 [default]`

Passing any CLI args bypasses all prompts and uses argparse as before.

---

11. Standalone executables (macOS + Windows)

User: I need this entire thing to be bundled into a single executable for both mac and windows so it can be used by people who are not as familiar with CLI interaction

Result: - Added a “Press Enter to exit…” pause at the end of `best_session.py` (only triggers when stdin is a real terminal, so piped/automated runs aren’t affected) — prevents the console window from vanishing instantly when double-clicked. - Built the **macOS** binary with PyInstaller: `pyinstaller --onefile --name pickleball-session-generator best_session.py` Output: `dist/pickleball-session-generator` (~8MB, no Python required). Verified both interactive and CLI-arg modes work from the compiled binary. - **Windows** build must happen on an actual Windows machine (PyInstaller doesn’t cross-compile). Instructions given: 1. Copy `pickleball_round.py`, `pickleball_session.py`, `best_session.py` to the Windows machine. 2. `pip install pyinstaller` 3. `pyinstaller --onefile --name pickleball-session-generator best_session.py` 4. Output: `dist\pickleball-session-generator.exe`

Final file layout

```
pickleball_round.py      — PickleballRound: generate(), __eq__/__hash__,
render_lines()/__repr__
```

```
pickleball_session.py    — PickleballSession: generate(), score(), __repr__
best_session.py          — CLI/interactive script: generate N sessions,
print the best
test_pickleball_round.py — quick script: generate + print one round
test_pickleball.py       — quick script: generate + print one session
pickleball-session-generator.spec — PyInstaller build spec (macOS)
dist/pickleball-session-generator — built macOS standalone executable
```